

Flutter Tools

[Link to the website](#)

1. Material library

Flutter > material.dart

Flutter package

LIBRARIES

animation

cupertino

foundation

gestures

material

painting

physics

rendering

scheduler

semantics

services

widget_preview

widgets

DART

dart:ui

dart:ui_web

CORE

dart:async

dart:collection

dart:convert

dart:core

dart:developer

dart:math

dart:typed_data

Search API Docs

material library

Flutter widgets implementing Material Design.

To use, import package:flutter/material.dart.

Using Material Design with Flutter

Google for Developers

Using Material Design with Flutter

Flutter

Watch on YouTube

See also:

- docs.flutter.dev/ui/widgets/material for a catalog of commonly-used Material component widgets.
- m3.material.io for the Material 3 specification
- m2.material.io for the Material 2 specification

Classes

AboutDialog

An about box. This is a dialog box with the application's icon, name, version number, and copyright, plus a button to show licenses for software used by the application.

AboutListTile

2. Scaffold class

[Flutter](#) > [material.dart](#) > Scaffold class

Scaffold class

material library

CLASSES

- AboutDialog
- AboutListTile
- AbsorbPointer
- AbstractLayoutBuilder
- Accumulator
- Action
- ActionChip
- ActionDispatcher
- ActionIconTheme
- ActionIconThemeData
- ActionListener
- Actions
- ActivateAction
- ActivateIntent
- Adaptation
- AdaptiveTextSelection...
- AlertDialog
- Align
- Alignment
- AlignmentDirectional
- AlignmentGeometry
- AlignmentGeometryT...
- AlignmentTween
- AlignTransition
- AlwaysScrollableScroll...
- AlwaysStoppedAnimat...

Implements the basic Material Design visual layout structure.

This class provides APIs for showing drawers and bottom sheets.

To display a persistent bottom sheet, obtain the [ScaffoldState](#) for the current [BuildContext](#) via [Scaffold.of](#) and use the [ScaffoldState.showBottomSheet](#) function.

This example shows a [Scaffold](#) with a [body](#) and [FloatingActionButton](#). The [body](#) is a [Text](#) placed in a [Center](#) in order to center the text within the [Scaffold](#). The [FloatingActionButton](#) is connected to a callback that increments a counter.

To create a local project with this code sample, run:

```
flutter create --sample=material.Scaffold.1 mysample
```

3. AppBar property

Flutter > material.dart > Scaffold > appBar property

Search API Docs

Scaffold class

CONSTRUCTORS

new

PROPERTIES

appBar

backgroundColor

body

bottomNavigationBar

bottomSheet

bottomSheetScrimBuil...

drawer

drawerBarrierDismissi...

drawerDragStartBeha...

drawerEdgeDragWidth

drawerEnableOpenDr...

appBar property

PreferredSizeWidget? appBar

(final)

An app bar to display at the top of the scaffold.

Implementation

final PreferredSizeWidget? appBar;

4. Body property

Flutter > material.dart > Scaffold > body property

Search API Docs

Scaffold class

CONSTRUCTORS

new

PROPERTIES

appBar

backgroundColor

body

bottomNavigationBar

bottomSheet

bottomSheetScrimBuil...

drawer

drawerBarrierDismissi...

drawerDragStartBeha...

drawerEdgeDragWidth

drawerEnableOpenDr...

drawerScrimColor

endDrawer

endDrawerEnableOpe...

extendBody

body property

Widget? body

(final)

The primary content of the scaffold.

Displayed below the `appBar`, above the bottom of the ambient `MediaQuery`'s `MediaQueryData.viewInsets`, and behind the `floatingActionButton` and `drawer`. If `resizeToAvoidBottomInset` is false then the body is not resized when the onscreen keyboard appears, i.e. it is not inset by `viewInsets.bottom`.

The widget in the body of the scaffold is positioned at the top-left of the available space between the app bar and the bottom of the scaffold. To center this widget instead, consider putting it in a `Center` widget and having that be the body. To expand this widget instead, consider putting it in a `SizedBox.expand`.

If you have a column of widgets that should normally fit on the screen, but may overflow and would in such cases need to scroll, consider using a `ListView` as the body of the scaffold. This is also a good choice for the case where your body is a scrollable list.

Implementation

final Widget? body;

5. bottomNavigationBar property

Flutter > material.dart > Scaffold > bottomNavigationBar property

Search API Docs

Scaffold class

CONSTRUCTORS

new

PROPERTIES

appBar

backgroundColor

body

bottomNavigationBar

bottomSheet

bottomSheetScrimBuil...

drawer

drawerBarrierDismissi...

drawerDragStartBeha...

drawerEdgeDragWidth

drawerEnableOpenDr...

drawerScrimColor

bottomNavigationBar property

Widget? bottomNavigationBar

(final)

A bottom navigation bar to display at the bottom of the scaffold.

Snack bars slide from underneath the bottom navigation bar while bottom sheets are stacked on top.

The `bottomNavigationBar` is rendered below the `persistentFooterButtons` and the `body`.

Implementation

final Widget? bottomNavigationBar;

6. SizedBox class

Flutter > widgets.dart > SizedBox class

widgets library

CLASSES
AbsorbPointer
AbstractLayoutBuilder
Accumulator
Action
ActionDispatcher
ActionListener
Actions
ActivateAction
ActivateIntent
Align
Alignment
AlignmentDirectional
AlignmentGeometry
AlignmentGeometryT...
AlignmentTween
AlignTransition
AlwaysScrollableScroll...
AlwaysStoppedAnima...
AndroidView
AndroidViewSurface
Animatable
AnimatedAlign
AnimatedBuilder
AnimatedContainer
AnimatedCrossFade
AnimatedDefaultText...
AnimatedFractionally...

SizedBox class

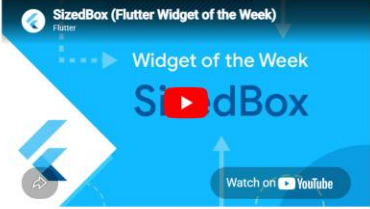
A box with a specified size.

If given a child, this widget forces it to have a specific width and/or height. These values will be ignored if this widget's parent does not permit them. For example, this happens if the parent is the screen (forces the child to be the same size as the parent), or another `SizedBox` (forces its child to have a specific width and/or height). This can be remedied by wrapping the child `SizedBox` in a widget that does permit it to be any size up to the size of the parent, such as `Center` or `Align`.

If either the width or height is null, this widget will try to size itself to match the child's size in that dimension. If the child's size depends on the size of its parent, the height and width must be provided.

If not given a child, `SizedBox` will try to size itself as close to the specified height and width as possible given the parent's constraints. If `height` or `width` is null or unspecified, it will be treated as zero.

The `SizedBox.expand` constructor can be used to make a `SizedBox` that sizes itself to fit the parent. It is equivalent to setting `width` and `height` to `double.infinity`.



This snippet makes the child widget (a Card with some Text) have the exact size 200x300, parental constraints permitting:

```
const SizedBox(  
  width: 200.0,  
  height: 300.0,  
  child: Card(child: Text('Hello World!')),  
)
```

7. StatefulWidget class

Flutter > widgets.dart > StatefulWidget class

widgets library

CLASSES
AbsorbPointer
AbstractLayoutBuilder
Accumulator
Action
ActionDispatcher
ActionListener
Actions
ActivateAction
ActivateIntent
Align
Alignment
AlignmentDirectional
AlignmentGeometry
AlignmentGeometryT...
AlignmentTween
AlignTransition
AlwaysScrollableScroll...
AlwaysStoppedAnima...
AndroidView
AndroidViewSurface
Animatable
AnimatedAlign
AnimatedBuilder
AnimatedContainer
AnimatedCrossFade
AnimatedDefaultText...
AnimatedFractionally...
AnimatedGrid
AnimatedGridState

StatefulWidget class

A widget that has mutable state.

State is information that (1) can be read synchronously when the widget is built and (2) might change during the lifetime of the widget. It is the responsibility of the widget implementer to ensure that the `State` is promptly notified when such state changes, using `State.setState`.

A `StatefulWidget` is a widget that describes part of the user interface by building a constellation of other widgets that describe the user interface more concretely. The building process continues recursively until the description of the user interface is fully concrete (e.g., consists entirely of `RenderObjectWidgets`, which describe concrete `RenderObjects`).

`StatefulWidget`s are useful when the part of the user interface you are describing can change dynamically, e.g. due to having an internal clock-driven state, or depending on some system state. For compositions that depend only on the configuration information in the object itself and the `BuildContext` in which the widget is inflated, consider using `StatelessWidget`.



`StatefulWidget` instances themselves are immutable and store their mutable state either in separate `State` objects that are created by the `createState` method, or in objects to which that `State` subscribes, for example `Stream` or `ChangeNotifier` objects, to which references are stored in final fields on the `StatefulWidget` itself.

The framework calls `createState` whenever it inflates a `StatefulWidget`, which means that multiple `State` objects might be associated with the same `StatefulWidget` if that widget has been inserted into the tree in multiple places. Similarly, if a `StatefulWidget` is removed from the tree and later inserted in to the tree again, the framework will call `createState` again to create a fresh `State` object, simplifying the lifecycle of `State` objects.

A `StatefulWidget` keeps the same `State` object when moving from one location in the tree to another if its creator used a `GlobalKey` for its `key`. Because a widget with a `GlobalKey` can be used in at most one location in the tree, a widget that uses a `GlobalKey` has at most one associated element. The framework takes advantage of this property when moving a widget with a global key from one location in the tree to another by grafting the (unique) subtree associated with that widget from the old location to the new location (instead of recreating the subtree at the new location). The `State` objects associated with `StatefulWidget` are grafted along with the rest of the subtree, which means the `State` object is reused (instead of being recreated) in the new location. However, in order to be eligible for grafting, the widget must be inserted into the new location in the same animation frame in which it was removed from the old location.

8. StatelessWidget class

Flutter > widgets.dart > StatelessWidget class

widgets library

CLASSES

absorbPointer

abstractLayoutBuilder

accumulator

Action

ActionDispatcher

ActionListener

actions

activateAction

activateIntent

align

alignment

alignmentDirectional

alignmentGeometry

alignmentGeometryT...

alignmentTween

alignTransition

alwaysScrollableScroll...

alwaysStoppedAnima...

androidView

androidViewSurface

animatable

animatedAlign

animatedBuilder

animatedContainer

animatedCrossFade

animatedDefaultText...

animatedFractionally...

animatedGrid

animatedGridState

animatedList

animatedListState

animatedModalBarrier

animatedOpacity

animatedPadding

StatelessWidget class

A widget that does not require mutable state.

A stateless widget is a widget that describes part of the user interface by building a constellation of other widgets that describe the user interface more concretely. The building process continues recursively until the description of the user interface is fully concrete (e.g., consists entirely of [RenderObjectWidgets](#), which describe concrete [RenderObjects](#)).

How to Create Stateless Widgets

Flutter Widgets 101

Stateless widget are useful when the part of the user interface you are describing does not depend on anything other than the configuration information in the object itself and the [BuildContext](#) in which the widget is inflated. For compositions that can change dynamically, e.g. due to having an internal clock-driven state, or depending on some system state, consider using [StatefulWidget](#).

Performance considerations

The build method of a stateless widget is typically only called in three situations: the first time the widget is inserted in the tree, when the widget's parent changes its configuration (see [Element.rebuild](#)), and when an [InheritedWidget](#) it depends on changes.

If a widget's parent will regularly change the widget's configuration, or if it depends on inherited widgets that frequently change, then it is important to optimize the performance of the [build](#) method to maintain a fluid rendering performance.

There are several techniques one can use to minimize the impact of rebuilding a stateless widget:

Minimize the number of nodes transitively created by the build method and any widgets it creates. For example, instead of an elaborate arrangement of [Rows](#), [Columns](#), [Paddings](#), and [SizedBoxes](#) to position a single child in a particularly fancy manner, consider using just an [Align](#) or a [CustomSingleChildLayout](#). Instead of an intricate layering of multiple [Containers](#) and with [Decorations](#) to draw just the right graphical effect, consider a single [CustomPaint](#) widget.

Use const widgets where possible, and provide a const constructor for the widget so that users of the widget can also do so.

Consider refactoring the stateless widget into a stateful widget so that it can use some of the techniques described at [StatefulWidget](#), such as caching common parts of subtrees and using [GlobalKeys](#) when changing the tree structure.

If the widget is likely to get rebuilt frequently due to the use of [InheritedWidgets](#), consider refactoring the stateless widget into multiple widgets, with the parts of the tree that change being pushed to the leaves. For example instead of building a tree with four widgets, the inner-most widget depending on the [Theme](#), consider factoring out the part of the build function that builds the inner-most widget into its own widget, so that only the inner-most widget needs to be rebuilt when the theme changes.

When trying to create a reusable piece of UI, prefer using a widget rather than a helper method. For example, if there was a function used to build a widget, a [State.setState](#) call would require Flutter to entirely rebuild the returned wrapping widget. If a [Widget](#) was used instead, Flutter would be able to efficiently re-render only those parts that really need to be updated. Even better, if the created widget is const, Flutter would short-circuit most of the rebuild work.

9. BoxFit enum

Flutter > painting.dart > BoxFit enum

painting library

CLASSES

accumulator

alignment

alignmentDirectional

alignmentGeometry

assetBundleImageKey

assetBundleImagePro...

assetImage

automaticNotchedSh...

beveledRectangleBor...

border

borderDirectional

borderRadius

borderRadiusDirectio...

borderRadiusGeometry

borderSide

boxBorder

boxDecoration

boxPainter

boxShadow

canvas

circleBorder

circularNotchedRecta...

clipContext

color

colorFilter

colorProperty

colorSwatch

continuousRectangle...

decoration

decorationImage

decorationImagePain...

BoxFit enum

How a box should be inscribed into another box.

See also:

applyBoxFit, which applies the sizing semantics of these values (though not the alignment semantics).

Inheritance

Object > Enum > BoxFit

Available extensions

EnumName

Values

fill → const BoxFit

Fill the target box by distorting the source's aspect ratio.

contain → const BoxFit

As large as possible while still containing the source entirely within the target box.

cover → const BoxFit

As small as possible while still covering the entire target box.

To actually clip the content, use [ClipBehavior: Clip.hardEdge](#) alongside this in a [FittedBox](#).

10. TextStyle class

Flutter > painting.dart > TextStyle class

Search API Docs

painting library

CLASSES

Accumulator

Alignment

AlignmentDirectional

AlignmentGeometry

AssetBundleImageKey

AssetBundleImagePro...

AssetImage

AutomaticNotchedSh...

BeveledRectangleBor...

Border

BorderDirectional

BorderRadius

BorderRadiusDirectio...

BorderRadiusGeometry

BorderSide

BoxBorder

BoxDecoration

BoxPainter

BoxShadow

Canvas

CircleBorder

CircularNotchedRecta...

ClipContext

Color

ColorFilter

ColorProperty

ColorSwatch

ContinuousRectangle...

TextStyle class

An immutable style describing how to format and paint text.



Bold

Here, a single line of text in a Text widget is given a specific style override. The style is mixed with the ambient DefaultTextStyle by the Text widget.

No, we need bold strokes. We need this plan.

```
const Text(  
  'No, we need bold strokes. We need this plan.',  
  style: TextStyle(fontWeight: FontWeight.bold),  
)
```

Flutter > painting.dart > TextStyle class

Search API Docs

painting library

CLASSES

Accumulator

Alignment

AlignmentDirectional

AlignmentGeometry

AssetBundleImageKey

AssetBundleImagePr...

AssetImage

AutomaticNotchedSh...

BeveledRectangleBor...

Border

BorderDirectional

BorderRadius

BorderRadiusDirectio...

BorderRadiusGeometry

BorderSide

BoxBorder

BoxDecoration

BoxPainter

BoxShadow

Canvas

CircleBorder

CircularNotchedRecta...

ClipContext

Color

ColorFilter

ColorProperty

ColorSwatch

ContinuousRectangle...

Decoration

DecorationImage

DecorationImagePain...

EdgeInsets

EdgeInsetsDirectional

EdgeInsetsGeometry

ExactAssetImage

FileImage

FittedSizes

FlutterLogoDecoration

FontFeature

Italics

As in the previous example, the Text widget is given a specific style override which is implicitly mixed with the ambient DefaultTextStyle.

Welcome to the present, we're running a real nation.

```
const Text(  
  'Welcome to the present, we're running a real nation.',  
  style: TextStyle(fontStyle: FontStyle.italic),  
)
```

Opacity and Color

Each line here is progressively more opaque. The base color is Colors.black, and Color.withOpacity is used to create a derivative color with the desired opacity. The root TextSpan for this RichText widget is explicitly given the ambient DefaultTextStyle, since RichText does not do that automatically. The inner TextStyle objects are implicitly mixed with the parent TextSpan's TextSpan.style.

If color is specified, foreground must be null and vice versa. color is treated as a shorthand for Paint().color = color.

If backgroundColor is specified, background must be null and vice versa. The backgroundColor is treated as a shorthand for background: Paint().color = backgroundColor.

You don't have the votes.
You don't have the votes!
You're gonna need congressional approval
and you don't have the votes!

```
RichText(  
  text: TextSpan(  
    style: DefaultTextStyle.of(context).style,  
    children: <TextSpan[  
      TextSpan(  
        text: 'You don't have the votes.\n',  
        style: TextStyle(color: Colors.black.withValues(alpha: 0.6)),  
      ),  
      TextSpan(  
        text: 'You don't have the votes!\n',  
        style: TextStyle(color: Colors.black.withValues(alpha: 0.8)),  
      ),  
      TextSpan(  
        text: 'You're gonna need congressional approval and you don't have the votes!\n',  
        style: TextStyle(color: Colors.black.withValues(alpha: 1.0)),  
      ),  
    ],  
  ),  
)
```

[More libraries](#)